1. Write a program to sort the array elements using Merge Sort Technique. Derive its time complexity

CODE:

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original array: ";
    printArray(arr, n);

    mergeSort(arr, 0, n - 1);

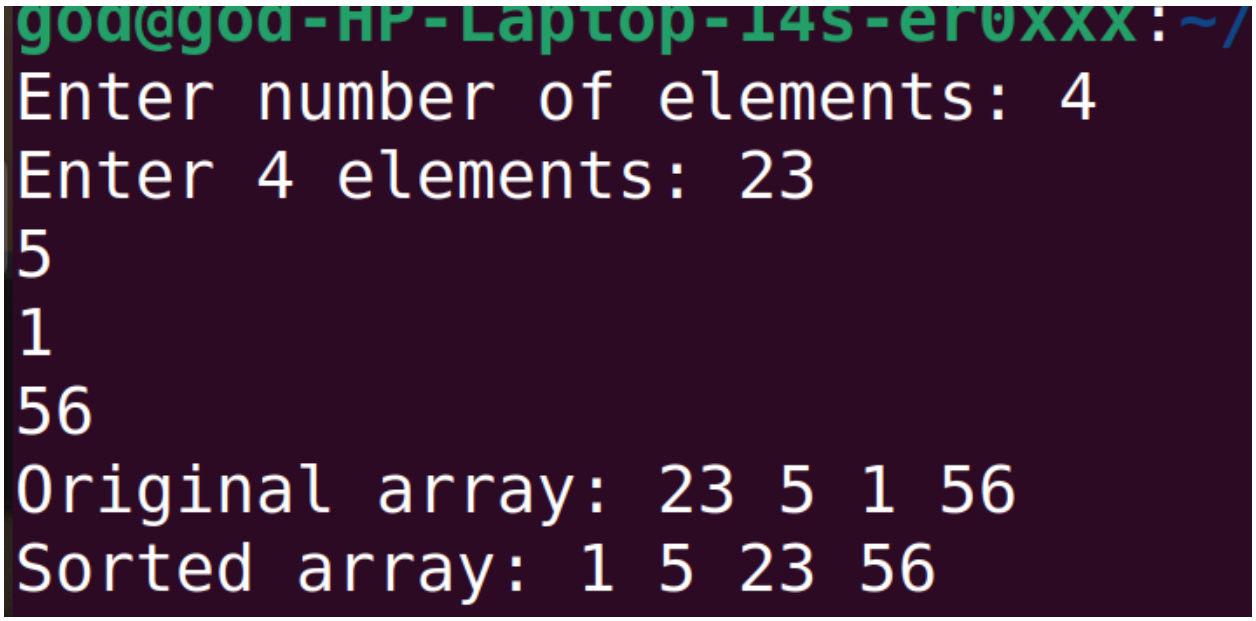
    cout << "Sorted array: ";
    printArray(arr, n);
```

```
    return 0;
}
```

TIME COMPLEXITY:

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

OUTPUT :



3. Write a program to sort the array elements using Quick Sort Technique.
Derive its time complexity

CODE:

```
#include <iostream>
using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original array: ";
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

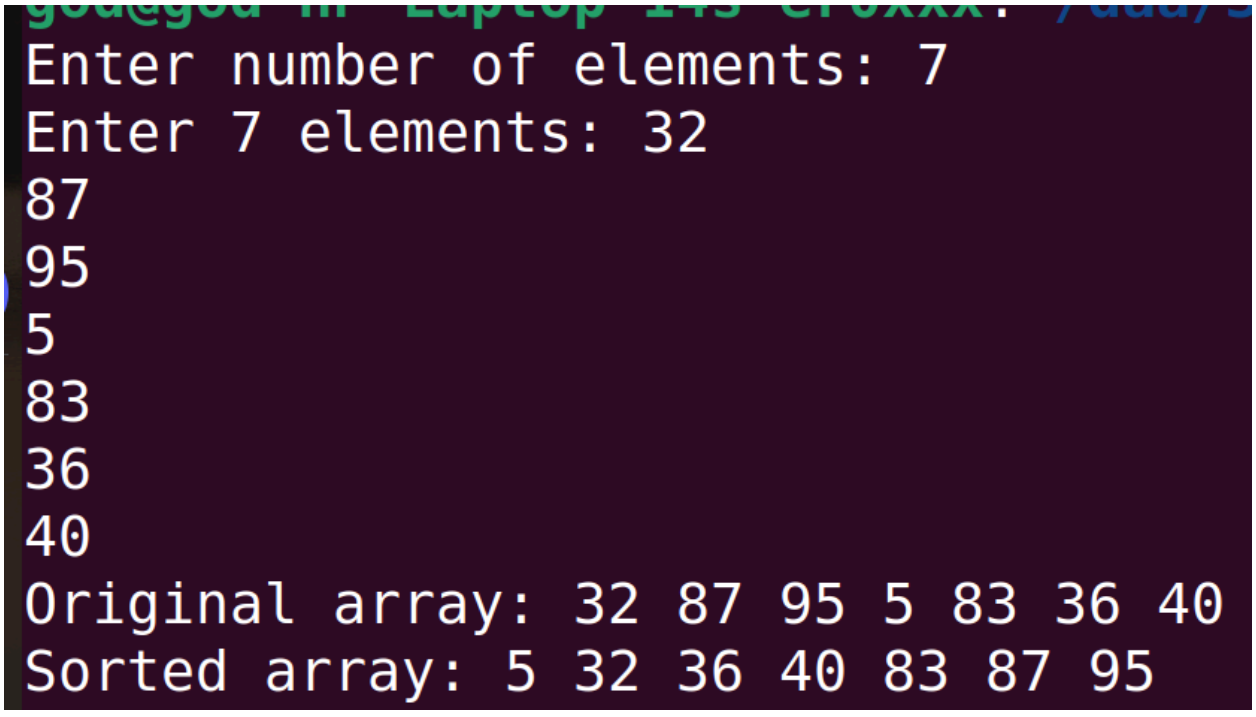
    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

TIME COMPLEXITY:

Case	Time Complexity	When it occurs
Best Case	O(n log n)	Pivot always divides array into equal halves
Average Case	O(n log n)	Random/typical input
Worst Case	$O(n^2)$	Already sorted/reverse sorted array + poor pivot choice (first/last element)

OUTPUT:



2. Write a program to sort the array elements using Bucket Sort Technique.
Derive its time complexity

ALGORITHM:

BUCKET-SORT(arr, n)

Input: arr[0..n-1] – array of n elements (assume values in range [0, 1)
or normalize integers/floats into this range)
Output: arr sorted in ascending order

1. Create n empty buckets: bucket[0], bucket[1], ..., bucket[n-1]

2. // Scatter: place each element into its appropriate bucket
   for i = 0 to n-1:
      index = floor(n * arr[i]) // bucket index for arr[i]
      insert arr[i] into bucket[index]

3. // Sort each bucket individually
   for i = 0 to n-1:
      sort bucket[i] // using Insertion Sort (or any sort)

4. // Gather: concatenate all sorted buckets back into arr
   index = 0
   for i = 0 to n-1:
      for each element x in bucket[i] (in sorted order):
         arr[index] = x
         index = index + 1

5. return arr

CODE:

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>
using namespace std;

void bucketSortIntegers(int arr[], int n) {
    if (n <= 0) return;

    int maxVal = *max_element(arr, arr + n);
    int minVal = *min_element(arr, arr + n);
    int range = maxVal - minVal + 1;

    int bucketCount = n;
    vector<int> buckets[bucketCount];

    for (int i = 0; i < n; i++) {
        int bucketIndex = (long long)(arr[i] - minVal) * bucketCount / range;
        buckets[bucketIndex].push_back(arr[i]);
    }

    for (int i = 0; i < bucketCount; i++)
        sort(buckets[i].begin(), buckets[i].end());

    int index = 0;
    for (int i = 0; i < bucketCount; i++)
        for (int val : buckets[i])
            arr[index++] = val;
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
```

```
        cin >> arr[i];

    bucketSortIntegers(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}
```

TIME COMPLEXITY:

Case	Complexity	Explanation
Best Case	O(n + k)	Elements are uniformly distributed — each bucket gets ~1 element
Average Case	O(n + k)	Assuming uniform distribution across k buckets
Worst Case	O(n²)	All elements fall into a single bucket → degrades to Insertion Sort on n elements

OUTPUT:

